

ML System Design Document

Credit Default Risk Scoring for Consumer Credit Card Origination

Course	DDM501 — AI in DevOps, DataOps, MLOps
Assignment	Individual Assignment 1 (5%)
Student	Võ Minh Sang — 25MS13286

Summary. A Vietnamese consumer finance company approves credit cards with a manual scorecard and a large team of underwriters. This document designs an ML system that estimates each applicant's probability of missing the next payment and turns it into one of three underwriting actions: APPROVE, REVIEW or DECLINE. The design covers requirements, a hierarchy of goals and metrics, the architecture and pipeline, and five design trade-offs. The feasibility figures come from a prototype trained on a dataset with the UCI "Default of Credit Card Clients" schema (30,000 customers, 23 features, 23.3% default rate). This dataset stands in for the company's own records, which are hypothetical.

1. Problem Definition

1.1 Context and background

The organisation is a mid-sized consumer finance company in Vietnam. It issues unsecured credit cards to salaried customers, mostly first-time borrowers aged 22–40. It receives about 6,000 card applications per business day through a mobile app, a partner network of electronics retailers and branch offices. Unsecured consumer credit is a high-margin, high-loss business. The portfolio's 90-day delinquency rate is around 6–8%. Charge-offs are the largest cost line after funding, so a small improvement in who gets approved has a large effect on profit.

1.2 Problem statement

The current process cannot reliably tell customers who will repay apart from customers who will not. It is also too slow and too expensive to scale. In measurable terms:

- **Loss:** about 22–23% of approved accounts miss at least one payment within the observation window. The current scorecard ranks risk with an estimated ROC AUC of about 0.68.
- **Cost and speed:** about 35% of applications go to manual review, and the median time to a decision is 26 hours. Partners at the retail point of sale need an answer in under a minute, or the customer walks away.
- **Consistency:** two underwriters looking at the same file agree only about 80% of the time. Decisions cannot be explained consistently to customers or to the regulator.

The goal is a service that returns a calibrated default probability and a decision in real time. It should reduce credit losses and manual workload without increasing unfair treatment between customer groups.

1.3 Current situation

Today a rule-based scorecard assigns points for age, income band, employment type and a credit bureau (CIC) flag. Any application that scores between two cut-offs, or that has an adverse CIC record, goes to an underwriter. The rules were last recalibrated two years ago. Nobody measures their performance systematically, and they use none of the behavioural data the company already holds for existing customers: repayment status, statement amounts and payments over the last six months.

1.4 Why machine learning

- Risk depends on **non-linear interactions**, for example credit utilisation combined with a recent payment delay. Hand-written rules capture these poorly, but gradient-boosted trees and regularised logistic regression with engineered ratios learn them from data.
- **Labelled history exists**, and every month of operation produces more of it. Each account eventually shows whether it defaulted, so the problem is standard supervised binary classification.
- The decision is **high-volume and repetitive**. The value lies in consistency and speed, which automated scoring provides and manual review does not.
- The environment **changes**, through marketing campaigns, the economic cycle and new partner channels. A model can be retrained and monitored for drift. A static scorecard silently decays.

1.5 Stakeholders

Stakeholder	Goals	Concerns
Applicants (end users)	A fast decision and fair treatment	Wrongful decline, opaque reasons, privacy of personal data
Risk management / CRO (business owner)	Lower credit losses at the same approval volume	Model risk, auditability, stability of the approval rate
Underwriting operations	A manageable review queue with good explanations	A sudden spike in REVIEW volume; cases arriving without supporting information
Sales and retail partners	High approval rate, instant answers at point of sale	Latency and downtime directly lose sales
Data science team	Accurate, reproducible models that can be improved	Label delay, data quality, training-serving skew
ML / platform engineering	A reliable, observable, easily deployed service	On-call load, rollback, dependency drift
Compliance and legal	Adherence to SBV lending rules and Decree 13/2023/ND-CP on personal data protection	Discrimination by sex or age, consent for data use, adverse-action explanations

2. Requirements Analysis

2.1 Functional requirements

ID	Requirement
FR1	Core ML function: estimate the probability that an applicant defaults on the next payment (binary classification, calibrated output in [0, 1]).
FR2	Map the probability to an action using two thresholds held in configuration, not in code: $< 0.30 \rightarrow$ LOW/APPROVE; $0.30-0.60 \rightarrow$ MEDIUM/REVIEW; $\geq 0.60 \rightarrow$ HIGH/DECLINE.
FR3	Provide a single-application endpoint (POST /predict) and a batch endpoint of up to 500 applications (POST /predict/batch) for nightly portfolio re-scoring and credit-limit reviews.
FR4	Return the top factors behind each decision (POST /explain, SHAP), used for underwriter review and adverse-action notices.
FR5	Expose /health and /model/info, the latter giving the model version, training date, metrics and the thresholds in force.
FR6	Reject malformed input at the edge with HTTP 422, for example an unknown education code, age outside 18–100 or a negative payment.
FR7	Log every request's features, score, decision and model version to support monitoring, audit and future labelling.

Input/output specification. The input is one JSON application with credit limit, demographics (sex, education, marital status, age) and three lists of six monthly values: repayment status (–2 to 8), statement amount and amount paid. The output is {default_probability, risk_band, decision, review_threshold, decline_threshold, model_version}. Both contracts are enforced by Pydantic schemas and published as OpenAPI.

Integration. The loan origination system (LOS) calls the scoring API synchronously over the internal network. Feature data for existing customers comes from the core banking system. Bureau data comes from the CIC gateway, and the LOS merges it before calling. Decisions and scores are written back to the LOS. Outcomes (default or no default) arrive from core banking 30–60 days later and are joined to the prediction log to create new training labels.

User interaction. Applicants see only the decision. Underwriters see the score, the risk band and the top explanatory factors in the review screen. Risk managers see monitoring dashboards and model reports. There is no direct user access to the model.

2.2 Non-functional requirements

Category	Requirement
Performance	p95 latency ≤ 100 ms and p99 ≤ 250 ms for /predict; batch of 500 ≤ 2 s; /explain p95 ≤ 1 s.
Throughput / load	Average about 0.2 requests/s (6,000 applications over a 9-hour day). Peak about 20 requests/s during promotions, plus nightly batch re-scoring of about 400,000 existing accounts in under one hour.
Scalability	A stateless API in containers scales horizontally behind a load balancer (Kubernetes HPA on CPU and request rate, 2–6 replicas). The model is loaded read-only at startup, so replicas are identical.
Reliability	99.9% monthly availability during business hours. Health checks verify that the model is loaded, not only that the process is alive. Graceful degradation: if the model is unavailable, the LOS falls back to the legacy scorecard and routes the case to REVIEW, never to an automatic approval. Rolling deployments allow instant rollback to the previous image and model version.
Maintainability	Scheduled monthly retraining, plus retraining triggered when drift is significant. Every model is versioned in a registry, and only one that passes a quality gate is promoted. Monitoring covers input drift (PSI), score distribution, decision mix, fairness gap and the four golden signals.
Security and privacy	TLS inside the network, no personal identifiers in the feature payload, role-based access to logs, data kept in-country, logs retained for 5 years for audit.

2.3 Data requirements

Aspect	Requirement
Sources	Core banking: limits, six months of repayment status, statements and payments. Application form: demographics. CIC bureau: external delinquency flags. Outcome labels: whether the customer defaulted in the following month.
Quality	A three-level validation gate runs before every training run. <i>Schema:</i> all 23 feature columns plus the target are present and numeric. <i>Statistics:</i> at least 5,000 rows, at most 2% missing per column, target rate between 5% and 60%. <i>Semantics:</i> categorical codes are valid, age is 18–100, repayment status is –2 to 8, and no payment is negative. Duplicate customer-month records are removed.
Privacy	Customer consent for credit scoring is recorded at application. Direct identifiers (name, national ID, phone) never enter the ML system, and records are keyed by a salted hash. Sex and age are used only for fairness monitoring , as a matter of policy. Access is logged and data minimisation is applied, as required by Decree 13/2023/ND-CP.
Volume	History of about 1.2 million account-months (roughly 300 MB as Parquet), growing by about 130,000 applications per month. The prototype uses 30,000 rows $\times$ 24 columns.
Labelling	The label delay is 30–60 days. Declined applicants never produce a label (selective labels), which is noted as a bias risk (Section 5).

3. Goals and Metrics

The goals form a hierarchy. Each model metric is chosen because it moves a system goal, and each system goal supports a business goal. Model metrics can be computed offline today. Business metrics lag by 1–3 months, which is why leading indicators are monitored as well.

Level	Goal	Metric	Target / threshold
Business	Reduce credit losses	Default rate of approved accounts at 3 months	–15% relative to the scorecard at equal approval rate
	Reduce operating cost	Share of applications sent to manual review	From 35% to $\leq 20\%$
	Grow approved volume	Point-of-sale decision time and approval rate	Decision < 1 min; approval rate not below the current level
System	Fast, available service	p95 latency, availability	≤ 100 ms; $\geq 99.9\%$
	Stable behaviour in production	Maximum feature PSI; DECLINE share	PSI < 0.10 stable, ≥ 0.25 act; DECLINE share $\leq 20\%$
	Safe releases	Share of models promoted only through the gate; rollback time	100%; < 5 min
Model	Rank risk well (primary)	ROC AUC on a held-out time-split	≥ 0.70 gate; target ≥ 0.75
	Find defaulters in an imbalanced population	PR AUC; recall at the review threshold of 0.30	PR AUC ≥ 0.45 ; recall ≥ 0.50
	Trustworthy probabilities	Brier score, calibration curve	Brier ≤ 0.15
	Fair treatment	Selection-rate gap between sexes at threshold 0.30	≤ 0.10 (hard gate)

Baselines. (1) The *prior* baseline predicts the base rate for everyone: ROC AUC 0.50, PR AUC 0.233. This is the floor. (2) The *legacy scorecard* has an estimated ROC AUC of about 0.68, which is the operational baseline to beat. (3) In the prototype, a *logistic regression on raw features* reaches ROC AUC 0.745 and PR AUC 0.545, and gradient boosting reaches 0.747–0.751. The target of ≥ 0.75 is therefore realistic. The quality gate of ≥ 0.70 blocks a regression without causing false alarms on normal run-to-run variation.

Why these metrics. Accuracy is excluded because a model that approves everyone scores 77% accuracy. ROC AUC measures ranking and is independent of the threshold, so it is the primary metric. PR AUC and recall reflect the minority class, which is where the losses are. The Brier score matters because the thresholds are applied to raw probabilities. The fairness gap is a **gate, not a report**: a model that improves AUC but widens the gap past 0.10 is rejected.

4. High-level Architecture Design

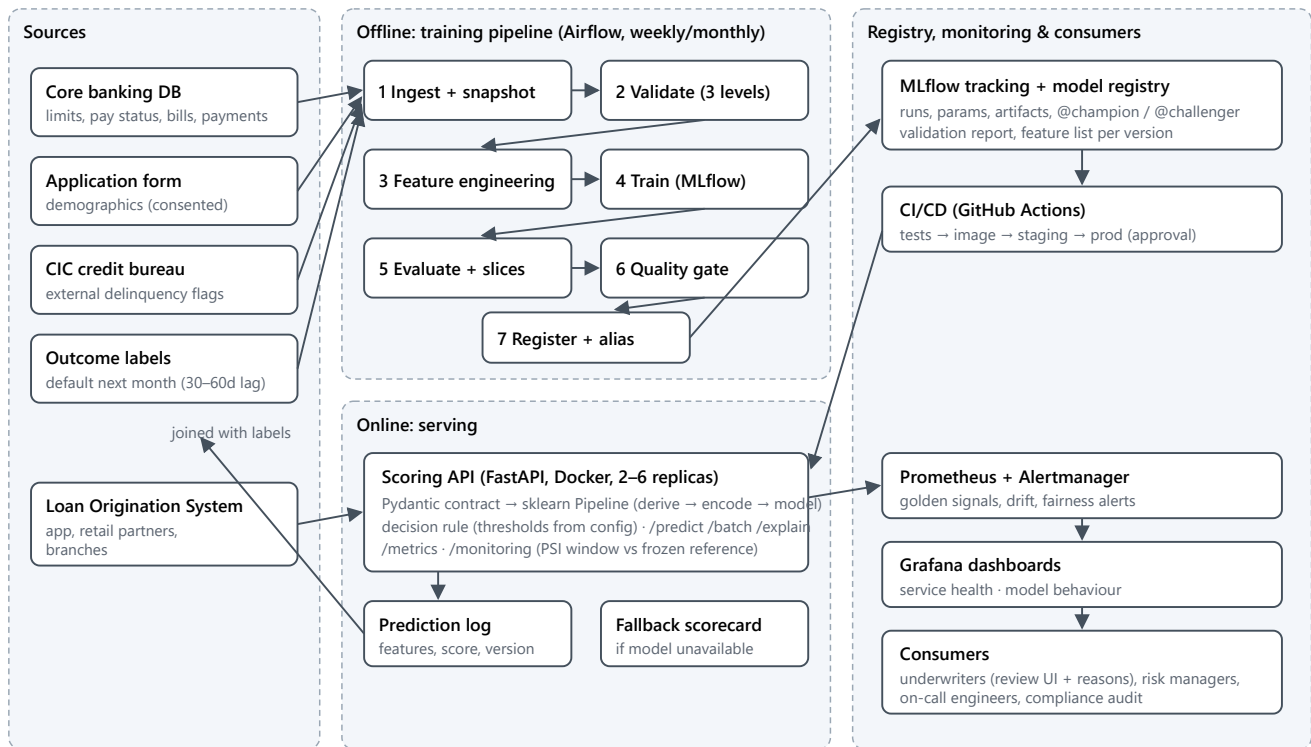


Figure 1. System architecture. The offline pipeline produces versioned, gated models; the online API serves the @champion version and exports monitoring signals. Prediction logs are joined with delayed labels to close the loop.

4.1 Data flow

- Online.** The LOS sends an application to `POST /predict`. The schema validates it. The fitted pipeline derives six features (utilisation, payment ratio, worst delay, months delayed, average bill, average payment), encodes and scales them, and scores. The decision rule maps the score to an action. The response goes back to the LOS, and the features, score and model version go to the prediction log and the in-memory drift window.
- Monitoring.** Every 10 s Prometheus scrapes `/metrics`: request rate, errors and latency, the score histogram, decision counts, PSI per feature against the *frozen training reference*, and selection rate by group. Alert rules page on-call staff for service failures and notify the risk team about drift and fairness.
- Offline.** Airflow runs the training DAG weekly, or on a drift alert: ingest a dated snapshot → validate → engineer features → train (every run tracked in MLflow) → evaluate, including per-group slices → quality gate → register. A model that passes the gate and beats the champion by ≥ 0.002 AUC becomes @champion; one that passes but does not beat it becomes @challenger; one that fails is registered and tagged *rejected*, which keeps an audit trail.
- Feedback.** Outcomes are joined to the prediction log after 30–60 days. They become new training data and also produce the lagging accuracy metrics that confirm or refute what the drift signals suggested.

4.2 Component description

Component	Purpose and responsibility	Technologies considered (chosen in bold)
Data ingestion	Extract a dated, immutable snapshot of the training table; make the train/test split reproducible (seeded and stratified).	Python/pandas + Parquet ; Spark if volume grows; DVC for snapshot versioning
Data validation	Block training on broken data; attach the validation report to every run.	Custom three-level checks ; Great Expectations / Pandera as alternatives
Feature engineering	Domain ratios and counts <i>inside</i> the sklearn Pipeline, so that training and serving share one code path and cannot skew.	scikit-learn Pipeline + ColumnTransformer ; a feature store (Feast) is not needed yet
Training and tracking	Fit candidate models and log parameters, data summary, artifacts and environment.	scikit-learn (LogReg, RF, HistGradientBoosting) , MLflow ; XGBoost/LightGBM as options
Evaluation and gate	Compute aggregate and per-group metrics; apply the written promotion rule automatically.	Custom module with thresholds held in config
Model registry	Version models; the aliases @champion and @challenger decide what is served.	MLflow Model Registry (aliases, not the deprecated stages)
Orchestration	Schedule and retry the pipeline, branch on the gate, clean up.	Apache Airflow ; Prefect or Kubeflow as alternatives
Serving API	Low-latency scoring with an input contract, health and model info endpoints.	FastAPI + Uvicorn in Docker ; BentoML / KServe as alternatives
Explainability	Give per-decision reasons to underwriters and for adverse-action notices.	SHAP TreeExplainer
Monitoring	Golden signals, drift, fairness, alerting and dashboards as code.	Prometheus, Alertmanager, Grafana ; Evidently for offline drift reports
CI/CD	Tests, linting, image build, staged deployment with approval.	GitHub Actions, GHCR , pre-commit

4.3 ML pipeline stages

Stage	Input → Output	Key checks
Data ingestion	Core banking tables → dated snapshot, train/test split	Row count, snapshot hash logged
Preprocessing	Raw frame → validated frame and validation report	Schema, statistics and semantics; fail fast
Feature engineering	23 raw columns → 29 features, one-hot encoded and scaled	Returns a copy; division by zero produces NaN, which is then imputed (never inf)
Training	Features + labels → fitted Pipeline and MLflow run	Fixed seeds; library versions logged
Evaluation	Model + test split → metrics, group metrics	AUC, PR AUC, recall, Brier, fairness gap
Inference	Application JSON → probability, band, decision	422 for invalid input; latency SLO; drift window

5. Trade-off Analysis

5.1 Accuracy vs latency (and explainability)

A large gradient-boosting ensemble, or a stack of models, might add 0.005–0.01 AUC. However, it costs latency and makes explanations harder. In the prototype, logistic regression with engineered features (ROC AUC 0.751) was at least as accurate as boosted trees (0.747–0.748) and scored in microseconds per row.

Decision: serve a single Pipeline model that is either regularised logistic regression or a moderate HistGradientBoosting model (depth ≤ 8). The p95 budget is 100 ms. Explanations are computed on demand through a separate endpoint, so their cost (about 10–100 ms) never slows down `/predict`.

5.2 Freshness vs cost

Retraining continuously or daily would track the population closely. However, labels arrive 30–60 days late, so daily retraining would mostly refit on the same labels at the cost of compute, validation and review.

Decision: retrain on a schedule (weekly to monthly) and also when drift is significant (PSI ≥ 0.25 sustained for 15 minutes). Drift is monitored in real time, but only a gated retrain changes the model. The drift reference stays frozen to the training data so that slow drift remains visible.

5.3 Simplicity vs performance

An online feature store, streaming features and a multi-model ensemble could extract more signal, but each adds infrastructure the team has to operate. **Decision:** use a monolithic sklearn Pipeline artifact with feature derivation inside it, one stateless API and batch retraining. This removes training–serving skew by construction and keeps rollback to a single alias change. More complexity is added only if the lagging business metric shows that the gap to the target cannot be closed otherwise.

5.4 Automation vs control

Fully automatic promotion could deploy a model that is more accurate but unfair, or one trained on a broken extract. **Decision:** automate the *checks* but keep humans on the *thresholds*. The gate (AUC ≥ 0.70 , PR AUC ≥ 0.45 , fairness gap ≤ 0.10 , where a missing metric fails) and the champion margin decide promotion; rejected models are still registered for audit; production deployment needs approval by a risk

officer through a protected GitHub environment; business thresholds are configuration owned by risk management. The model never auto-approves when it is down.

5.5 Privacy vs personalisation

Adding more personal data, such as phone metadata, social data or precise location, would probably improve AUC but raises consent, legal and fairness risks under Decree 13/2023/ND-CP. **Decision:** use only credit-relevant behavioural data that was collected with consent. Sensitive attributes (sex, age) are held for *monitoring* fairness, and whether to drop them as model inputs is reviewed with compliance. Identifiers are hashed, and personal data never leaves the country. The model's AUC is accepted as the price of a defensible feature set.

5.6 Known risks

- **Selective labels:** declined applicants never show outcomes, so the training data comes only from past approvals. Mitigation: a small randomised review band, and reject-inference analysis.
- **Feedback loops and drift:** marketing campaigns change the applicant population (for example younger customers with lower limits). This is mitigated by PSI per feature, score quantiles and decision-mix alerts.
- **Accuracy vs fairness:** the most accurate family (logistic regression) had the widest selection-rate gap (0.054 against 0.028–0.037 for tree models). The gate makes this trade-off explicit rather than accidental.

References: Kästner, Le Goues & Hilton (2024), *Machine Learning in Production*; Huyen (2022), *Designing Machine Learning Systems*; Sculley et al. (2015), *Hidden Technical Debt in ML Systems*; Yeh & Lien (2009), UCI Default of Credit Card Clients; Government of Vietnam, Decree 13/2023/ND-CP on personal data protection.